SeeFire

Otonom İç Mekan Yangın Algılama Robotu

Paralel Çalışma Planı

Kişi Bazlı Adım Adım Yol Haritası

Rapor v2.0 ile uyumlu

Ders	CSE 396 — Bilgisayar Mühendisliği Projesi
Üniversite	Gebze Teknik Üniversitesi
Dönem	2025–2026 Bahar
Takım	Grup 4

Ekip ve Renk Anahtarı

Badge	Ad Soyad	Öğrenci No	Birincil Modüller
BEKİR	Bekir Emre Sarıpınar	220104004039	M2 (Motor Kontrol) + M3 desteği
ALPEREN	Alperen Şahin	220104004943	M1 (Şasi) + M3 (HC-SR04, I2C)
BATUHAN	Batuhan Yanar	220104004059	M7 (Veri Loglama) + M3 (SPI/MQ-2)
EMRE	Emre Can Tuncer	200104004115	M4 (Görü) + M6 (Karar Motoru)
SEMİH	Semih Sarıkoca	220104004038	M6 (Karar Motoru) + M4 desteği
FURKAN	Ahmet Furkan Arslan	220104004044	M5 (Navigasyon) + M6 entegrasyon
YUSUF	Yusuf Alperen Çelik	220104004064	M5 (Navigasyon) + M7 desteği
HALİL	Halil Buğra Şen	230104004088	M1 (Şasi) + M6 alarm/demo

Adım durumu renkleri:

HEMEN BAŞLANABİLİR Laptop/dizüstüde, donanım olmadan hemen başlanabilir.

BEKLE: ÖRNEK Başlamadan önce belirtilen adım(lar)ın tamamlanması gerekiyor.

DONANIM GEREKLİ Fiziksel robot veya Raspberry Pi zorunlu.

Temel kurallar:

1. **Arduino yok.** Tüm motor PWM, encoder ve sensör I/O doğrudan Raspberry Pi GPIO üzerinden çalışıyor.
2. **config.py tek kaynak.** Her sabit (pin no, eşik, dosya yolu) config.py'ye girer ve oradan import edilir. Modül içinde asla hardcode edilmez.
3. **Mock Mode zorunlu.** Her modül donanım olmadan, laptop'ta çalışabilmelidir. Mock testleri geçmeyen bir adım tamamlanmış sayılmaz.
4. **MCP3208 12-bit (0–4095).** Eski rapordaki 1023 değeri yanlıştı. Tüm kod ve testlerde 4095 kullanılır.
5. **Batarya eşikleri 2S 18650:** Uyarı 6.8 V, kritik 6.4 V. Eski 3S değerleri (9.6 V, 8.4 V) geçersiz.

İçindekiler

Ekip ve Renk Anahtarı **1**

1 Genel Faz Haritası **3**

2 Faz 1 — Temel Altyapı (Donanımsız Paralel Başlangıç) **4**
2.1 Batuhan — M7 Veri Loglama Modülü 4
2.2 Yusuf — M7 Persistence ve M5 Mock Başlangıcı 5
2.3 Emre — M4 Görü Boru Hattı ve M6 FSM İskeleti 5
2.4 Semih — M4 Engel Yön Kararı ve M6 Geçiş Mantığı 7
2.5 Furkan — M5 Engel Kaçınma ve M6 Batarya Mantığı 7
2.6 Bekir — M2 Sınıf İskeleti (Mock) 9
2.7 Alperen ve Halil — Faz 1'de Fiziksel Hazırlık 10

3 Faz 2 — Şasi Montajı ve Güç Kurulumu **11**

4 Faz 3 — Donanım Sürücüleri: M2 ve M3 **13**
4.1 İz A: M2 Motor Kontrolü ve Encoder Odometrisi — Bekir 13
4.2 İz B: M3 Sensör Entegrasyonu — Alperen, Batuhan 14

5 Faz 4 — Navigasyon ve Engel Kaçınma Fiziksel Testi **17**

6 Faz 5 — Karar Motoru M6 Entegrasyonu **19**

7 Faz 6 — Kalibrasyon, Test ve Demo Hazırlığı **21**

8 Kişi Bazlı Özet — Kim Ne Zaman Ne Yapıyor **23**

9 Toplantı Takvimi Özeti **25**

10 Bağımlılık Özet Tablosu **27**

11 Teknik Hatırlatmalar **29**

1. Genel Faz Haritası

Proje altı fazdan oluşuyor. Her fazdaki bağımsız izler **eş zamanlı** yürütülür. Kritik yol: **M7 → M2+M3 → M5 → M6**.

Faz	Ad	Çalışanlar	Geçiş Koşulu (bir sonraki faza girebilmek için)
1	Temel Altyapı	**BATUHAN** **YUSUF** **EMRE** **SEMİH**	M7 testleri geçiyor; M4 laptop'ta çalışıyor; M5 ve M6 mock kodu yazılıyor
2	Şasi ve Güç	**FURKAN** **ALPEREN** **HALİL** + faz 1 devam	Pi açılıyor, 5 V doğrulandı, encoder monte edildi
3	Donanım Sürücüleri	**BEKİR** **ALPEREN** **BATUHAN**	Robot 50 cm hareket ediyor; 3 sensör veri döndürüyor
4	Navigasyon	**FURKAN** **YUSUF** (+ **BEKİR** destek)	3 sektör rotası engelsiz tamamlanıyor, 6 snapshot tetikleniyor
5	Karar Motoru	**EMRE** **SEMİH** **FURKAN** **HALİL**	Uçtan uca ALARM döngüsü çalışıyor, SQLite'a yazılıyor
6	Kalibrasyon ve Demo	Tüm 8 kişi	3 ardışık başarılı demo koşusu

Boşluk doldurma prensibi: Bir adımda başkasını beklemeniz gerekiyorsa, ilerideki adımlarınızdan bağımlılığı olmayanları öne çekin. Her kişinin yol haritasında bu durum açıkça işaretlenmiştir.

2. Faz 1 — Temel Altyapı (Donanımsız Paralel Başlangıç)

Donanım gerekmez. Bu fazdaki tüm çalışmalar laptop'ta yapılır.
Kim çalışır: **BATUHAN** **YUSUF** **EMRE** **SEMİH** **FURKAN** (beşi eş zamanlı, birbirinden bağımsız)

ALPEREN ve **HALİL** bu fazda fiziksel montaj planlaması yapar. **BEKİR** bu fazda M2 sınıf iskeletini yazar.

2.1 Batuhan — M7 Veri Loglama Modülü

BATUHAN bu modülde tamamen bağımsızdır. Hiçbir şeyi beklemeden başlar ve bitirir.

Adım 2.1 — Ortam kurulumu

HEMEN BAŞLANABİLİR

Repoyu laptop'a klonla. Python 3.11+ ve `pytest`'in kurulu olduğunu doğrula. `SEEFIRE_DATA_DIR` ortam değişkenini geçici bir klasöre yönlendir (örn. `/tmp/seefire`). `m7_logging/` klasörünün var olduğunu kontrol et.

Adım 2.2 — SQLite şeması ve WAL modu

HEMEN BAŞLANABİLİR

`m7_logging/logging.py` içinde `init()` fonksiyonunu yaz: SQLite veritabanını aç/oluştur, `PRAGMA journal_mode=WAL` çalıştır, `events` tablosunu oluştur. Tüm DB işlemlerini `threading.Lock()` altına al.

Tablo alanları: `id` (PK, otomatik), `timestamp` TEXT, `event_type` TEXT, `fusion_score` REAL, `sensor_data` TEXT, `snapshot_path` TEXT.

Adım 2.3 — log_event() ve get_events() implementasyonu

HEMEN BAŞLANABİLİR

`log_event(event: m7_event_t) → int`: satır ekle, otomatik ID'yi döndür.
`get_events(event_type=None, limit=100) → list[dict]`: isteğe bağlı tip filtresi ve limit ile sorgu yap.

Adım 2.4 — M7 birim testlerini çalıştır

HEMEN BAŞLANABİLİR

`pytest m7_logging/tests/test_logging.py -v` komutunu çalıştır. 12 testin tamamı yeşil olmalı: WAL modu, concurrent yazma, atomik map yazma, snapshot dizini oluşturma dahil.

Tamamlanma kriteri: Hiçbir test kırmızı değil.

2.2 Yusuf — M7 Persistence ve M5 Mock Başlangıcı

YUSUF önce M7'nin persistence kısmını bitirir, ardından beklemek zorunda kalmadan M5'in mock yazımına başlar.

Adım 2.5 — save_map() — atomik yazma

HEMEN BAŞLANABİLİR

`save_map(json_str: str)`: önce `.tmp` uzantılı geçici dosyaya yaz, sonra `os.replace()` ile hedef konuma taşı (işlemsel yazma — yarım yazma riski yok). Test: yazma sırasında process öldürme simülasyonu yap, hedef dosya bozulmamış olmalı.

Adım 2.6 — load_map() ve save_snapshot()

HEMEN BAŞLANABİLİR

`load_map() → str | None`: dosya yoksa `None` döndür, varsa içeriği döndür.
`save_snapshot(jpeg_bytes: bytes, event_id: int) → str`: `config.SNAPSHOT_DIR`'i otomatik oluştur, dosyayı `{event_id}_{UTC_timestamp}.jpg` olarak kaydet, tam yolu döndür.

Adım 2.7 — M5 NavigationController sınıf iskeleti — Mock

HEMEN BAŞLANABİLİR

Faz 3 tamamlanana kadar fiziksel test yapılamaz, ama kodun tamamı şimdi yazılabilir.

`m5_navigation/navigation.py`: `NavigationController(snapshot_callback)` sınıfını oluştur. `run(waypoints)` metodunda sektör döngüsünü yaz. `_check_midpoint(sector_id)`: encoder toplam mesafesi sektör orta noktasına eşit veya büyükse ve henüz geçilmediyse `_snapshot()` çağır. `_snapshot(label)`: `m2.stop()`, 0.3 s bekle, `snapshot_callback(label)`, 0.2 s bekle.

Mock modda `m2.get_total_distance_cm()` çağrısı her iterasyonda `STEP_DISTANCE_CM` kadar artır.

Adım 2.8 — M5 mock uçtan uca test

HEMEN BAŞLANABİLİR

`NavigationController(snapshot_callback=print).run()` komutunu laptop'ta çalıştır. 3 sektörde tam 6 snapshot tetiklenmeli (her sektörde 1 orta nokta + 1 waypoint). Encoder sayacının her hedef değere ulaştığını log ile doğrula.

2.3 Emre — M4 Görü Boru Hattı ve M6 FSM İskeleti

EMRE iki bağımsız işi paralel yürütür: M4'ü laptop'ta tamamlar, ardından M6 iskeletini yazar.

Adım 2.9 — M4 geliştirme ortamı

HEMEN BAŞLANABİLİR

`pip install ultralytics opencv-python numpy Pillow pytest` komutunu çalıştır. YOLOv8n modelini indir: `from ultralytics import YOLO; model = YOLO('yolov8n.pt')`. Roboflow'dan yangın/duman veri setini bul, 5–10 test video klibi indir (yangın var, duman var, yangın yok).

Adım 2.10 — M4 YOLOv8n çıkarım boru hattı

HEMEN BAŞLANABİLİR

`m4_vision/vision.py`: `VisionM4.init()`: kamerayı aç (`cv2.VideoCapture(0)`), ilk 5 kareyi at (otomatik pozlama). Arka plan thread'ini oluştur: kare al (320×320) → normalize et → YOLOv8n INT8 çıkarımı → güven eşiğini geç → mutex korumalı result struct'a yaz.

Mock modda (kamera yok): sabit bir sahteci detection döndür; thread açılıp kapanmalı, hata vermemeli.

Adım 2.11 — M6 FSM durum makinesi iskeleti

HEMEN BAŞLANABİLİR

`m6_decision/decision.py`: durum enum'unu tanımla (INIT, NAVIGATE, VERIFY, ALARM). `m6_init()` ile tüm modül referanslarını başlat. `m6_run()`: 500 ms periyodik döngü. `m6_stop()`: `threading.Event` ile döngüyü nazikçe kapat.

Bu adımda gerçek sensör/motor çağrısı yok; sahte verilerle durum geçişlerini konsolda yazdır.

Adım 2.12 — M6 füzyon skoru hesaplama

HEMEN BAŞLANABİLİR

`m6_calculate_fusion_score(vision_result, fusion_data) → float`:

$$\text{skor} = 0.5 \times \text{vision_conf} + 0.3 \times \frac{\text{smoke_level}}{4095} + 0.2 \times \text{clamp}\left(\frac{\text{ir_temp} - 40}{60}, 0, 1\right)$$

Payda 4095'tir (12-bit MCP3208). Birim testi yaz: YOLO=0.80, smoke=2000/4095, IR=75°C → beklenen skor 0.664.

Adım 2.13 — M4 Pi performans testi

BEKLE: Faz 2 Adım 2.5 (Pi açık ve çalışıyor)

Pi hazır olduğunda: M4 kodunu Pi'ye kopyala, `pip install` bağımlılıkları, C270 kamerayı bağla. `python -m m4_vision.vision` çalıştır, FPS ölç. Hedef: ≥3 FPS, 320×320, INT8 modeli.

FPS düşükse: çözünürlüğü 256×256'ya indir, INT8 quantization aktif mi kontrol et.

2.4 Semih — M4 Engel Yön Kararı ve M6 Geçiş Mantığı

SEMİH **EMRE** 'den bağımsız olarak M4'ün ikinci sorumluluğunu ve M6 geçiş mantığını yazar.

Adım 2.14 — M4 kamera engel yön algoritması

HEMEN BAŞLANABİLİR

`determine_turn_direction(frame=None) → Optional[str]`: kareyi gri'ye çevir → GaussianBlur (5×5) → Canny kenar tespiti (50, 150) → alt yarıya odaklan (ROI) → en soldaki ve en sağdaki kenar sütununu bul → boşlukları karşılaştır → `"SAĞ"` veya `"SOL"` döndür. Engel görünmüyorsa `None` döndür.

Statik test: farklı engel fotoğrafları üzerinde test et, kararın tutarlı olduğunu doğrula.

Mock mod (OpenCV yok): direkt `None` döndür.

Adım 2.15 — M6 durum geçiş mantığı

HEMEN BAŞLANABİLİR

`fsm_transition(state, vision, sensors, score) → State`: tüm geçiş kurallarını kodla.

- NAVIGATE → VERIFY: `vision_conf > 0.5` VEYA `smoke_alert` VEYA `ir_temp > 55`
- VERIFY → ALARM: `skor > 0.7`
- VERIFY → NAVIGATE: `skor < 0.4` (yanlış alarm, logla)
- ALARM → NAVIGATE: `reset_alarm()` çağrısı
- Herhangi → STOP: `batarya_v < BATTERY_CRIT_V`

Her geçiş için birim testi yaz; sahte giriş verileriyle tüm yolları doğrula.

Adım 2.16 — M6 mock FSM entegrasyon testi

BEKLE: Adım 2.8 (M6 FSM iskeleti, EMRE)

EMRE FSM iskeletini bitirince: sahte sensör verileriyle tüm durumları tetikle. INIT → NAVIGATE → VERIFY → ALARM → NAVIGATE zincirini çalıştır. Yanlış alarm yolunu test et. Her geçiş konsolda doğru loglanmalı.

Bu arada beklerken: M4 görü boru hattı testlerine yardım et (**EMRE** , Adım 2.7).

2.5 Furkan — M5 Engel Kaçınma ve M6 Batarya Mantığı

FURKAN M5'in en kritik parçasını mock olarak yazar; Faz 3 tamamlanana kadar beklediği sürede M6'nın batarya mantığını tamamlar.

Adım 2.17 — M5 PositionVerifier — verify_start()

HEMEN BAŞLANABİLİR

`m5_navigation/position.py`: `PositionVerifier.verify_start()`: `m3.get_navigation_sensors_` çağır, `left_cm` ve `right_cm`'yi `START_LEFT_CM` ve `START_RIGHT_CM` ile karşılaştır. Tolerans aşılırsa gerçek ve beklenen değerleri içeren açıklayıcı `RuntimeError` fırlat.

Mock modda: sensörler rastgele ama makul değer döndürür, tolerans geçilmez.

Adım 2.18 — M5 PositionVerifier — verify_and_correct()

HEMEN BAŞLANABİLİR

`verify_and_correct()`: filtrelenmiş sol sensörü oku, `START_LEFT_CM`'den sapmayı hesapla. Sapma toleransı aşıyorsa: sola kaydıysa sağa dön + `FINE_TUNE_STEP_CM` ileri git + sola dön. Sağa kaydıysa tam tersi. Düzeltme yönünü ve miktarını logla.

Adım 2.19 — M5 ObstacleAvoidance — yön kararı ve yan geçiş

HEMEN BAŞLANABİLİR

`m5_navigation/obstacle.py`:

`_decide_direction()`: önce `m4_vision.determine_turn_direction()` çağır. `None` dönerse `m3.get_navigation_sensors_filtered()` ile sol/sağ mesafeyi karşılaştır.

`_side_pass(sector_id) → float`: `m3.left_cm < OBSTACLE_CLEAR_CM` olduğu sürece: `midpoint_callback(sector_id)` çağır (detour sırasında orta nokta geçildiyse snapshot tetikle), `m2.drive_distance_cm(SIDE_STEP_CM)`, `traveled` sayacını artır. 200 cm güvenlik sınırı. Toplam yan mesafeyi döndür.

Adım 2.20 — M5 ObstacleAvoidance — encoder ile geri dönüş

HEMEN BAŞLANABİLİR

`avoid(sector_id)`:

1. Kuzey mesafe sayacını kaydet: `north_dist = m2.get_total_distance_cm()`
2. Yön karar: `_decide_direction()`
3. 90° seçilen yöne dön
4. `_side_pass()` → yan mesafeyi al
5. Geri dönüş sırası (sadece encoder, sensör kullanılmaz):
   - 90° dön (kuzeye bak) → 90° dön (rota tarafına bak)
   - `drive_distance_cm(yan_mesafe)` → 90° dön (kuzeye bak)
6. Sayacı geri yükle: `m2.set_total_distance_cm(north_dist)`
7. `position_verifier.verify_and_correct()`

Adım 2.21 — M6 batarya izleme mantığı

HEMEN BAŞLANABİLİR

M6 ana döngüsünde, her iterasyonda `m2.get_battery_voltage()` çağır. `BATTERY_CRIT_V` (6.4 V) altındaysa: `m2.stop()`, `m7.log_event(LOW_BATTERY)`, döngüden çık. `BATTERY_LOW_V` (6.8 V) altındaysa ama kritik üstündeyse: yalnızca uyarı logla, çalışmaya devam et.

Bu adım EMRE'nin FSM iskeletini beklemez. Mantık ayrı bir fonksiyon olarak yaz, FSM iskeletiyle sonra birleştirilir.

Adım 2.22 — M5 mock uçtan uca entegrasyon testi

HEMEN BAŞLANABİLİR

YUSUF'un Adım 1.7'deki `NavigationController` koduyla **FURKAN**'ın `ObstacleAvoidance` kodunu birleştir. Mock modda çalıştır: rota ortasında sahte bir engel tetikle (`center_cm`'yi 10'a indir). Engel kaçınmanın tüm adımlarını (karar, dönüş, yan geçiş, geri dönüş, düzeltme) konsol loglarıyla doğrula. 6 snapshot hâlâ tetiklenmeli.

2.6 Bekir — M2 Sınıf İskeleti (Mock)

BEKİR bu fazda donanım olmadan M2'nin tüm mantık yapısını yazar.

Adım 2.23 — M2 sınıf yapısı ve yön tablosu

HEMEN BAŞLANABİLİR

`m2_motor/motor.py`: `MotorM2` sınıfını oluştur. Constructor'da `MOCK_MODE` bayrağını belirle (RPi.GPIO yoksa True). Yön GPS tablosunu docstring olarak belgele:

- FORWARD: IN1=H IN2=L IN3=H IN4=L
- TURN_LEFT: IN1=L IN2=H IN3=H IN4=L
- TURN_RIGHT: IN1=H IN2=L IN3=L IN4=H
- STOP: tümü LOW, PWM duty=0

Mock modda tüm GPIO çağrıları konsola log yazar.

Adım 2.24 — drive_distance_cm() ve turn mantığı — Mock

HEMEN BAŞLANABİLİR

`drive_distance_cm(cm)`: `_reset_encoder_window()` çağır, motorları başlat. Mock modda: `elapsed` × `MOCK_CM_PER_SEC` ile mesafeyi hesapla. Hedefi geçince `stop()`, `_total_distance_cm += cm`.

`turn_left_90()` ve `turn_right_90()`: `MOCK_TURN_90_SECONDS` kadar bekle, dur.

`get_total_distance_cm()` ve `set_total_distance_cm(v)`: getter/setter.

2.7 Alperen ve Halil — Faz 1'de Fiziksel Hazırlık

ALPEREN ve **HALİL** bu fazda yazılım yazmaz; malzeme alımını ve fiziksel planlama çalışmalarını tamamlar.

Adım 2.25 — Malzeme kontrolü ve eksik sipariş

HEMEN BAŞLANABİLİR

ALPEREN : BOM (malzeme listesi) ile tüm paketleri karşılaştır. Eksik varsa hemen sipariş ver. Özellikle dikkat: 3 adet HC-SR04, encoder disk seti, 1k/2k/20k/10k dirençler, MCP3208 DIP-16, LM2596, XT60 pigtail, rocker switch, 8.4 V şarj adaptörü.

Adım 2.26 — Montaj planı ve kablo planı çizimi

HEMEN BAŞLANABİLİR

HALİL : Şasi üzerinde sensör yerleşimini ölçekli olarak çiz (el çizimi veya basit diyagram). HC-SR04 ön (merkez, 50 mm), sol (sol taraf, 50 mm, 45°), sağ (sağ taraf, 50 mm, 45°), MLX90614 (üst merkez, 80 mm), MQ-2 (üst ön, 60 mm), kamera (üst ön, 90 mm, 10° yukarı). Encoder disk konumlarını işaretle.

Faz 1 Kapanış Toplantısı — Çevrimiçi (Discord) **BATUHAN** **YUSUF** **EMRE** **SEMİH** **FURKAN** **BEKİR**

Gündem:

1. **BATUHAN** : M7 testlerini ekran paylaşımıyla canlı çalıştır. 12 test yeşil.
2. **YUSUF** : M5 mock uçtan uca testi göster (6 snapshot konsol logu).
3. **EMRE** : M6 FSM iskeletini demo yap; füzyon skoru birim testini çalıştır.
4. **SEMİH** : M6 geçiş birim testlerini göster.
5. **FURKAN** : Engel kaçınma mock logunu göster; sayaç geri yüklemeyi doğrula.
6. **BEKİR** : `drive_distance_cm()` mock modda 50 cm simüle ettiğini göster.

Süre: 45 dakika. Tüm demo'lar geçerse Faz 2'ye başlanır.

3. Faz 2 — Şasi Montajı ve Güç Kurulumu

Kim çalışır: **ALPEREN** **HALİL** (ağırlıklı olarak). **EMRE** bu fazda M4 Pi testini yapar (Adım 2.9).
Faz 1 ile paralellik: Faz 1 bitmeden de başlanabilir (tamamen bağımsız fiziksel işler).

Adım 3.1 — Şasi çerçeve montajı

DONANIM GEREKLİ

ALPEREN : Çift katlı akrilik şasi kitini monte et. 4 adet TT motoru (FL=0, FR=1, RL=2, RR=3) braketlere tak. Tekerlekleri geçir. Elle döndürerek dört tekerin de serbest döndüğünü doğrula. Ölçümleri kontrol et: dingil arası 140 mm, iz genişliği 120 mm.

Adım 3.2 — Encoder disk kurulumu

DONANIM GEREKLİ

HALİL : Sol ve sağ motor millerine encoder disk montajını yap. IR optocoupler sensörün disk slotundan sürtünmesiz geçtiğini kontrol et. Sol ve sağ encoder kablolarını etiketle (karışmasın). Raspberry Pi'nin oturacağı konuma doğru yönlendir.

Adım 3.3 — Buck converter ön kontrolü (KRİTİK)

DONANIM GEREKLİ

ALPEREN : **Raspberry Pi'yi bağlama.** 2S bataryayı LM2596'nın girişine bağla. Trimmeri döndürerek çıkışı tam 5.0 V'a ayarla. 60 saniye boyunca multimetreyle ölç (kararlı olmalı). Sadece bu adım tamamlandıktan sonra Pi'yi bağla.

Adım 3.4 — Güç dağıtımı ve Raspberry Pi montajı

DONANIM GEREKLİ

ALPEREN **HALİL** : Batarya XT60 → rocker switch → L298N Vcc ve LM2596 girişini paralel olarak bağla. LM2596 çıkışını (USB-C, 5 V) Raspberry Pi'ye bağla. Pi'yi üst deste üzerine vida veya bantla sabitle. Heatsink ve fanı Pi CPU'suna yapıştır. Pi'yi aç; aynı WiFi'daki laptop'tan SSH ile bağlantıyı doğrula.

Adım 3.5 — Sensör braket montajı

DONANIM GEREKLİ

HALİL : Tüm sensörleri Faz 1'deki montaj planına göre yerlere sabitle: HC-SR04 ön/sol/sağ, MLX90614, MQ-2, kamera. Kablolar breadboard'a yönlendiriliyor olmalı; kablo güzergahlarını zipbant ile düzenle.

Adım 3.6 — Breadboard: HC-SR04 gerilim bölücüler

DONANIM GEREKLİ

ALPEREN : Her HC-SR04 Echo pini için bir gerilim bölücü kur (3 adet):

- Echo pin → 1kΩ → kavşak → Pi GPIO
- Kavşak → 2kΩ → GND

Her bölücüyü multimetreyle doğrula: Echo HIGH (5 V) iken kavşak 3.33 V okumalı. ÖN, SOL, SAĞ olarak etiketle. GPIO pinlerini config.py'ye göre bağla: ECHO_FRONT=20, ECHO_LEFT=24, ECHO_RIGHT=8.

Adım 3.7 — Breadboard: batarya gerilim bölücüsü

DONANIM GEREKLİ

HALİL : Batarya izleme bölücüsünü kur:

- Batarya+ → 20kΩ → kavşak → MCP3208 CH1
- Kavşak → 10kΩ → GND

7.4 V bataryada kavşak ≈2.47 V okumalı (MCP3208'in 3.3 V referansı altında).

Adım 3.8 — M4 Pi performans testi

BEKLE: Adım 2.4 (Pi açık ve çalışıyor)

EMRE : M4 kodunu Pi'ye kopyala, bağımlılıkları yükle, C270 kamerayı bağla. `python -m m4_vision.vision` çalıştır, FPS ölç. Hedef: ≥3 FPS.

FPS düşükse: çözünürlüğü 256×256'ya indir veya INT8 quantization kontrolü yap.

Faz 2 Kapanış Toplantısı — Yüz yüze (lab) **ALPEREN** **HALİL** **EMRE**

Gündem:

1. Pi SSH ile açılıyor ve erişilebilir durumda.
2. Buck converter çıkışı 5.0 V (multimetre ile doğrula).
3. Tüm sensör braketleri yerli yerinde.
4. 3 gerilim bölücü doğru voltaj veriyor.
5. **EMRE** M4 FPS benchmarkını Pi'de canlı gösteriyor.

Süre: 60 dakika.

4. Faz 3 — Donanım Sürücüleri: M2 ve M3

Bloklayan: Faz 2 kapanış toplantısı (Pi açık, breadboard kablolu).
İki bağımsız iz paralel yürür.

4.1 İz A: M2 Motor Kontrolü ve Encoder Odometrisi — Bekir

Adım 4.1 — L298N GPIO kablolaması ve init_hardware()

DONANIM GEREKLİ

BEKİR : L298N'i Pi GPIO'ya bağla (config.py'e göre): IN1=17, IN2=18, IN3=27, IN4=22, ENA=12 (PWM), ENB=13 (PWM). Motor çıkışlarını bağla: kanal A → sol motorlar (FL+RL paralel), kanal B → sağ motorlar (FR+RR paralel). L298N Vcc'yi batarya rayından besle (Pi'den değil). **Açmadan önce multimetreyle tüm bağlantıları kontrol et.**

`init_hardware()`'i tamamla: `GPIO.setmode(BCM)`, IN1-IN4'ü çıkış olarak yapılandır, ENA/ENB'yi 1 kHz PWM olarak başlat, encoder pinlerini (GPIO 6 sol, GPIO 21 sağ) pull-down girişi olarak yapılandır, yükselen kenar interrupt callback'lerini kaydet.

Adım 4.2 — Temel hareket testi

DONANIM GEREKLİ

BEKİR : Robotu yüksekte tut (tekerlekler yerde değil). `motor_drive("forward", 60)` çağır, 1 saniye bekle. 4 tekerin de ileri döndüğünü doğrula. Geri, sol dönüş, sağ dönüş varyantlarını test et. Multimetreyle: ENA/ENB duty cycle hız parametresiyle orantılı mı doğrula.

Adım 4.3 — drive_distance_cm() Pi entegrasyonu

DONANIM GEREKLİ

BEKİR : Faz 1'de yazılan mock logic'i gerçek encoder interrupt'larıyla entegre et. `_measured_distance_since_window_cm()`: gerçek encoder tick sayısını `ENCODER_TICKS_PER_CM`'e böl. Mock mod hâlâ çalışmalı (encoder yoksa elapsed-time fallback).

Adım 4.4 — Encoder kalibrasyonu

DONANIM GEREKLİ

BEKİR : Robotu yere koy, karşısına mezura koy. `drive_distance_cm(100)` çağır. Gerçek mesafeyi ölç. `ENCODER_TICKS_PER_CM`'i ayarla: yeni değer = mevcut değer × (100/gerçek mesafe). 5 kez tekrarla, ±2 cm içinde tutarlı olana kadar. Kalibre edilen değeri config.py'e yaz.

Adım 4.5 — 90° dönüş kalibrasyonu

DONANIM GEREKLİ

BEKİR : `turn_right_90()` çağır, gerçek dönüş açısını iletki veya zemine bant ile ölç. `MOCK_TURN_90_SECONDS`'ı dönüş ±5° içinde olana kadar ayarla. Kalibre edilen değeri config.py'e yaz.

Adım 4.6 — Alarm çıkışları

DONANIM GEREKLİ

BEKİR : LED'i (GPIO 26, seri 330Ω) ve buzzer'ı (GPIO 19) breadboard'a bağla. `set_alarm(True, True)` çağır → LED yanar ve buzzer çalar. `set_alarm(False, False)` → ikisi de söner.

Adım 4.7 — Batarya izleme entegrasyonu

BEKLE: Adım 3.10 (MCP3208 SPI hazır)

BEKİR : M3 SPI sürücüsü hazır olduğunda `get_battery_voltage()`'ı tamamla: `m3_sensors.read_battery_adc()` çağır, gerilim bölücü formülünü uygula: $V = (\text{ADC}/4095) \times 3.3 \times 3$. Multimetre okumalarıyla karşılaştır. Uyarı ve kritik eşik loglarını doğrula.

Bu adımı beklerken: Adım 3.14 (M2 birim testleri) ve Adım 3.15 (M3 entegrasyon test desteği) işlerini yap.

Adım 4.8 — M2 birim testleri

HEMEN BAŞLANABİLİR

BEKİR (Adım 3.10'u beklerken yapılabilir): Mock modda drive_distance_cm, turn sıralaması ve alarm kontrolü için birim testleri yaz. Gerçek GPIO call'ları `unittest.mock.patch` ile çıkar. Test: 50 cm komutu → encoder sayacı 50 artıyor, stop çağrılıyor, total_distance_cm 50 artıyor.

4.2 İz B: M3 Sensör Entegrasyonu — Alperen, Batuhan

Adım 4.9 — HC-SR04 sürücüsü (3 sensör)

DONANIM GEREKLİ

ALPEREN : `_read_ultrasonic(trig_pin, echo_pin) → float`: TRIG 10 µs HIGH, ECHO HIGH bekle (50 ms timeout), HIGH süresini ölç, `süre_µs / 58.0` döndür. 400 cm üzerinde veya timeout'ta −1.0 döndür. 3 sensörü ayrı ayrı test et: eline farklı mesafelerden tut (10, 30, 60 cm), okumaları doğrula. Sensörler sıralı tetiklenmeli (eş zamanlı değil, çapraz konuşma riski var).

Adım 4.10 — Medyan filtreli navigasyon okuma

DONANIM GEREKLİ

ALPEREN : `get_navigation_sensors_filtered(samples=3)` → `NavData`: her eksen için N okuma al (okumalar arası 20 ms), her eksen için `median()` uygula. Test: okuma sırasında sensör kablosuna hafifçe dokun (gürültü enjekte et), medyan değerin stabil kaldığını doğrula.

Adım 4.11 — MCP3208 SPI sürücüsü

DONANIM GEREKLİ

BATUHAN : Pi'de SPI'yi etkinleştir: `sudo raspi-config` → `Arayüz Seçenekleri` → `SPI` → `Etkinleştir`. MCP3208'i bağla: MOSI=GPIO10, MISO=GPIO9, SCLK=GPIO11, CS=GPIO5. MQ-2 analog çıkışını CH0'a, batarya gerilim bölücüsünü CH1'e bağla.

`_read_mcp3208(channel)` → `int`: 3 baytlık SPI transferi, 12-bit sonucu çözümle (0–4095). Doğrulama: MQ-2'yi temiz havada oku (∼100–300 beklenir), CH1'den batarya değerini oku ve voltaj formülüyle çevir, multimetreyle karşılaştır.

Adım 4.12 — MQ-2 ısınma ve füzyon verisi

DONANIM GEREKLİ

BATUHAN : `get_fusion_sensors()` → `FusionData`: CH0'ı oku, `start_time`'dan 60 saniye geçti mi kontrol et. Geçmediyse `smoke_alert = False` (ısınma süresinde asla alarm vermez). Geçtiyse `smoke_alert = (smoke_level > SMOKE_THRESHOLD)`. MQ-2'yi 60 saniye ısıt, okumaları her 5 saniyede logla, stabilize olduğunu doğrula.

Adım 4.13 — MLX90614 I2C sürücüsü

DONANIM GEREKLİ

ALPEREN : Pi'de I2C'yi etkinleştir: `sudo raspi-config` → `Arayüz Seçenekleri` → `I2C` → `Etkinleştir`. MLX90614'ü bağla: SDA=GPIO2 (pin 3), SCL=GPIO3 (pin 5), 3.3 V, GND. Kablolar **20 cm'den kısa** olmalı. `i2cdetect -y 1` komutunu çalıştır, `0x5A` adresi görünmeli. Her okumayı `try/except` içine al; hata durumunda son önbellek değerini döndür. Sıcak ve soğuk yüzeylere yönelterek okumaların değiştiğini doğrula.

Adım 4.14 — M3 entegrasyon testi

DONANIM GEREKLİ

ALPEREN **BATUHAN** : `python test_mocks.py`'i Pi'de **donanım bağlı** olarak çalıştır (Mock Mode değil). Doğrulanacaklar:

- `get_navigation_sensors()`: sol/merkez/sağ mesafeleri mantıklı değerler.
- `get_fusion_sensors()`: smoke_level ve ir_temp değerleri geliyor.
- `read_battery_adc()`: ölçülen batarya voltajıyla örtüşüyor.

Faz 3 Kapanış Toplantısı — Yüz yüze (lab) **BEKİR** **ALPEREN**

BATUHAN

Canlı demo: Robot yerde 100 cm ilerliyor (±2 cm). 3 HC-SR04 mesafe döndürüyor. MQ-2 çakmaklığa tutulduğunda 300'ü geçiyor. IR sensör oda sıcaklığını okuyor. Batarya voltajı multimetreyle örtüşüyor. LED ve buzzer komutla aktif oluyor.
Süre: 90 dakika.

5. Faz 4 — Navigasyon ve Engel Kaçınma Fiziksel Testi

Bloklayan: Faz 3 kapanış toplantısı (robot hareket ediyor, sensörler çalışıyor).
Kim çalışır: **FURKAN** **YUSUF** (**BEKİR** destek için hazır bekler).

M5 API Gözden Geçirme — Çevrimiçi (Discord) **FURKAN** **YUSUF** **BEKİR** **ALPEREN**

Faz 4'e başlamadan yapılacak. **BEKİR** ve **ALPEREN** M2 ve M3 genel API'lerini anlatır. **FURKAN** ve **YUSUF** anladıklarını kendi sözleriyle açıklar. Kopyalama yok: M5 içinde hiçbir sensör veya motor mantığı tekrar yazılmaz.
Süre: 30 dakika.

Adım 5.1 — verify_start() fiziksel testi

DONANIM GEREKLİ

FURKAN : Robotu bilinen bir başlangıç noktasına yerleştir. `verify_start()`'ın geçtiğini doğrula. Robotu 15 cm yana kaydır ve `RuntimeError` fırlattığını doğrula. config.py'de `START_LEFT_CM` ve `START_RIGHT_CM`'yi gerçek duvar mesafelerine göre güncelle.

Adım 5.2 — Tek sektör traversali testi

DONANIM GEREKLİ

YUSUF : Koridor veya odada 100 cm'lik bir güzergah hazırla. `config.WAYPOINTS = [(100, 1)]` olarak ayarla. `NavigationController.run()` çalıştır. Gözlemle: robot sektör orta noktasında (50 cm) durdu mu, snapshot callback çağrıldı mı, waypoint'te (100 cm) durdu mu. Mesafe toleransı: ±5 cm.

Adım 5.3 — 3 sektör tam rota testi

DONANIM GEREKLİ

YUSUF **FURKAN** : 300 cm'lik güzergah hazırla. `WAYPOINTS = [(100,1),(200,2),(300,3)]`. Tam rotayı çalıştır. Doğrula: toplam 6 snapshot (3 orta nokta + 3 waypoint), her stop noktası ±5 cm içinde, rota boyunca pozisyon sapması birikmemiş.

Adım 5.4 — Engel kaçınma fiziksel testi

DONANIM GEREKLİ

FURKAN : Sektör 2'nin ortasına karton kutu yerleştir. Tam rotayı çalıştır. Doğrula:

- Robot engeli fark ediyor (`center_cm < OBSTACLE_THRESHOLD_CM`).

- Kamera yön kararını veriyor (veya HC-SR04 fallback devreye giriyor).
- Yan geçiş tamamlanıyor.
- Encoder ile rotaya geri dönüş yapılıyor.
- Sektör 2 orta nokta snapshot'ı detour sırasında bile tetikleniyor.
- Robot rotaya döndükten sonra doğru kuzeye yönelmiş.
- Sektör 2 waypoint snapshot'ı tetikleniyor.
- Sektör 3 normal tamamlanıyor.

Adım 5.5 — Pozisyon düzeltme ince ayarı

DONANIM GEREKLİ

FURKAN : Engel kaçınma sonrası pozisyon doğrulamasının aşırı düzeltme yapmadığını kontrol et. Gerekirse `FINE_TUNE_STEP_CM`'i veya `POSITION_TOLERANCE_CM`'i ayarla. Robot engel sonrası ±5 cm içinde rotaya dönüyor olmalı.

Faz 4 Kapanış Toplantısı — Yüz yüze (lab) **FURKAN** **YUSUF** **BEKİR**

Canlı demo: Engelli 3 sektör rotası. 6 snapshot callback çağrılıyor (konsolda görünüyor). Robot tüm stop noktalarında ±5 cm içinde. Engel kaçınma sonrası detour'da sektör orta nokta snapshot'ı tetikleniyor. **BEKİR** herhangi bir M2 sorunu için hazır bekliyor.
Süre: 90 dakika.

6. Faz 5 — Karar Motoru M6 Entegrasyonu

Bloklayan: Faz 4 kapanış + Faz 2'de M4 Pi testi.
Kim çalışır: EMRE SEMİH FURKAN HALİL

M6 API Gözden Geçirme — Çevrimiçi (Discord) EMRE FURKAN HALİL

Faz 5'e başlamadan yapılacak. FURKAN ile EMRE `snapshot_callback` arayüzünü netleştirir: M6 callback'i sağlar, M5 callback'i çağırır, M6 M5 thread'ini çalıştırır. Kilitleme modeli ve hangi thread'den ne çağrılır kararlaştırılır.
Süre: 30 dakika.

Adım 6.1 — ALARM aksiyon zinciri

BEKLE: Adım 1.3 (M7 log_event) + B2.7 (M4 capture_frame)

HALİL: `m6_trigger_alarm(fusion_score, sensor_data)`'ı tamamla:

1. `m4.capture_frame()` → JPEG baytları al
2. `m7.log_event(FIRE_DETECTED, ...)` → event ID al
3. `m7.save_snapshot(jpeg_bytes, event_id)`
4. `m2.set_alarm(True, True)`

4 adımın toplamının 50 ms'yi geçmediğini zamanlayarak doğrula.

Adım 6.2 — M5'i M6 NAVIGATE durumuna bağlama

BEKLE: Adım 2.8 (M6 FSM iskeleti) + Faz 4 tamamlandı

FURKAN: INIT → NAVIGATE geçişinde `NavigationController(snapshot_callback=m6_snaps` ayrı bir thread'de başlat. `m6_snapshot_handler(label: str)`'ı uygula: `m4.capture_frame()` → `m7.log_event(SECTOR_SNAPSHOT)` → `m7.save_snapshot()`.

Adım 6.3 — FSM ve füzyon skoru mock entegrasyon testi

BEKLE: Adım 2.10 (fsm_transition birim testi)

SEMİH EMRE: Mock modda tüm durumları sahte sensör girdileriyle tetikle. Doğrula: INIT → NAVIGATE, NAVIGATE → VERIFY (vision_conf=0.6 enjekte et), VERIFY → ALARM (skor=0.8), ALARM M7 SQLite'a yazılıyor, JPEG dosyası oluştu. Yanlış alarm yolu: skor=0.2 enjekte et → NAVIGATE'e dön.

Adım 6.4 — Batarya izleme M6 entegrasyonu

BEKLE: Adım 3.13 (get_battery_voltage() hazır)

HALİL: Faz 1'de **FURKAN**'ın yazdığı batarya mantığını FSM ana döngüsüne entegre et. Düşük pil simülasyonu yap (sahte değer enjekte et), kritik durumda döngünün durduğunu doğrula.

Adım 6.5 — Tam fiziksel entegrasyon testi

BEKLE: Adım 5.1 + 5.2 + 5.3 + 5.4 tamamlandı

EMRE **SEMİH** **FURKAN** **HALİL**: `main.py`'i fiziksel robot üzerinde çalıştır. Gözlemle:

- Pi açılıyor, M7 SQLite oluşturuluyor, tüm modüller init oluyor.
- M3 sensörler (MQ-2 ısınma süresini test için atla veya bekle).
- M5 navigasyon başlıyor; sektör snapshot'ları `/data/snapshots/`'a yazılıyor.
- Ateş uyaranı (çakmak MQ-2'ye yakın, sıcak nesne MLX90614'e yakın, yangın videosu kameraya göster).
- ALARM durumu: LED yanar, buzzer çalar, JPEG kaydedildi, SQLite kaydı oluştu.
- Manuel reset: NAVIGATE'e dönüş.

Faz 5 Kapanış Toplantısı — Yüz yüze (lab) **EMRE** **SEMİH** **FURKAN** **HALİL** **BEKİR** **BATUHAN**

Canlı demo: Tam ALARM döngüsü fiziksel robot üzerinde. Veritabanı sorgusu canlı gösterilir:
`sqlite3 /data/seefire.db "SELECT * FROM events ORDER BY id DESC LIMIT 10;"`
Snapshot klasöründe JPEG dosyası görünüyor. Tüm 6 sektör snapshot'ı ve FIRE_DETECTED event'i kayıtlı.
Süre: 90–120 dakika. Bu, final demo provası.

7. Faz 6 — Kalibrasyon, Test ve Demo Hazırlığı

Bloklayan: Faz 5 kapanış toplantısı.
Kim çalışır: Tüm 8 kişi.

Adım 7.1 — Encoder son kalibrasyonu

DONANIM GEREKLİ

BEKİR : Demo ortamında (gerçek zemin, gerçek yüzey) encoder kalibrasyonunu tekrarla. `drive_distance_cm(100)`'ı 3 kez çalıştır, her seferinde gerçek mesafeyi ölç. ±2 cm tolerans içinde değilse `ENCODER_TICKS_PER_CM`'i güncelle. Aynısını dönüş kalibrasyonu için yap.

Adım 7.2 — START_LEFT_CM ve START_RIGHT_CM kalibrasyonu

DONANIM GEREKLİ

ALPEREN : Robotu demo başlangıç noktasına tam olarak yerleştir. `m3.get_navigation_sensors_f` çağır. Dönen `left_cm` ve `right_cm` değerlerini `config.START_LEFT_CM` ve `config.START_RIGHT_CM` olarak güncelle.

Adım 7.3 — WAYPOINTS demo ortamına göre güncelleme

DONANIM GEREKLİ

BATUHAN : Demo güzergahının gerçek uzunluğunu ölç. `config.WAYPOINTS`'i demo odasına göre güncelle. Sektör orta noktalarının yangın algılamanın anlamlı olduğu konumlara denk geldiğini doğrula.

Adım 7.4 — YOLO güven eşiği ince ayarı

DONANIM GEREKLİ

EMRE : Demo odası aydınlatmasında M4 çıkarımını test et: (a) yangın videosu gösteren telefon, (b) kırmızı nesne (yanlış pozitif olmamalı). `VISION_CONF_THRESHOLD`'ı ve gerekirse `FUSION_ALARM_THRESH`'i ayarla.

Adım 7.5 — MQ-2 duman eşiği ince ayarı

DONANIM GEREKLİ

BATUHAN : 60 saniyelik ısınma sonrası temiz havadaki baseline'ı kaydet. Çakmaklık dumanıyla tetiklenme değerini kaydet. `SMOKE_THRESHOLD`'ı baseline üstü ama duman altında tutacak şekilde ayarla.

Adım 7.6 — Stres testi: 3 ardışık demo koşusu

DONANIM GEREKLİ

SEMİH : Pi'yi yeniden başlatmadan 3 tam demo koşusu yap. Koşular arası yalnızca `m6_reset_alarm()` ve `m6_run()` çağır. 3 koşunun tamamında: hata yok, tüm snapshot'lar kaydedildi, tüm event'ler SQLite'da. Bellek veya zamanlama sorununu **EMRE** 'ye bildir.

Adım 7.7 — Demo senaryosu ve kuru çalıştırma

DONANIM GEREKLİ

HALİL **YUSUF** : Tek sayfalık demo senaryosu yaz: (1) robot başlangıca konuluyor, (2) `python main.py` çalıştırılıyor, (3) robot sektörleri geziyor, (4) yangın uyaranı veriliyor, (5) ALARM tetikleniyor, (6) SQLite canlı gösteriliyor. Senaryoyu tam olarak takip ederek bir kuru çalıştırma yap. Süreyi ölç (hedef: $\leq$5 dakika).

Adım 7.8 — Engel kaçınma demo varyantı (isteğe bağlı)

DONANIM GEREKLİ

FURKAN : Sektör 2'ye karton kutu koy, engel kaçınmalı demo varyantını hazırla. Detour sırasında sektör orta nokta snapshot'ının hâlâ tetiklendiğini doğrula. Bu, izleyiciler için güçlü bir ek etki yaratır.

Final Demo Prova Toplantısı — Yüz yüze (demo odası) Tüm 8 kişi

Gündem:

1. Demo senaryosunu iki kez tam olarak çalıştır.
2. Her koşuyu zaman tut.
3. Tüm görsel çıktıların (LED, buzzer) izleyiciye görünür olduğunu doğrula.
4. SQLite sorgusunu yansıtılmış laptop ekranında göster.
5. Kim hangi bölümü anlatıyor — rolleri paylaş.
6. Demo sonrası teknik soru-cevap için her kişi kendi modülünü anlatmaya hazır olsun.

Süre: 90 dakika.

8. Kişi Bazlı Özet — Kim Ne Zaman Ne Yapıyor

Bu tablo her kişinin **gerçek çalışma sırasını** gösterir. Bekleyiş boşlukları doldurulmuştur.

Kişi	Faz	Ne yapıyor	Bloklayan / Not
BEKİR	1	M2 sınıf iskeleti, yön tablosu, drive_distance_cm mock mantığı	Bağımsız
BEKİR	1	M2 birim testleri (mock)	Bağımsız
BEKİR	2	(bekler, M2 kodu hazır)	Donanım bekleniyor
BEKİR	3	L298N GPIO kablolaması, init_hardware()	Faz 2
BEKİR	3	Temel hareket testi, encoder entegrasyonu	Adım 3.1
BEKİR	3	Encoder kalibrasyonu (100 cm)	Adım 3.3
BEKİR	3	90° dönüş kalibrasyonu	Adım 3.4
BEKİR	3	Alarm çıkışları (LED, buzzer)	Adım 3.1
BEKİR	3	Batarya izleme (MCP3208 hazır olduğunda)	Adım 3.10
BEKİR	4	Faz 4'te destek	Hazır bekler
BEKİR	6	Encoder son kalibrasyonu (demo ortamı)	Faz 5
ALPEREN	1	Malzeme kontrolü, eksik sipariş	Bağımsız
ALPEREN	2	Şasi çerçeve montajı	Donanım
ALPEREN	2	Buck converter ön kontrolü (KRİTİK)	Adım 2.2
ALPEREN	2	Güç dağıtımı, Pi montajı	Adım 2.3
ALPEREN	2	HC-SR04 gerilim bölücüler (3 adet)	Adım 2.4
ALPEREN	3	HC-SR04 sürücüsü (3 sensör)	Faz 2
ALPEREN	3	Medyan filtreli okuma	Adım 3.9
ALPEREN	3	MLX90614 I2C sürücüsü	Adım 2.4
ALPEREN	3	M3 entegrasyon testi	Adım 3.9+3.12+3.13
ALPEREN	6	START_LEFT/RIGHT kalibrasyonu	Faz 5
BATUHAN	1	M7 SQLite şeması, WAL modu	Bağımsız
BATUHAN	1	log_event(), get_events()	Adım 1.1
BATUHAN	1	M7 birim testleri (12 test)	Adım 1.2-1.3
BATUHAN	3	MCP3208 SPI sürücüsü (12-bit)	Faz 2
BATUHAN	3	MQ-2 ısınma, FusionData	Adım 3.10
BATUHAN	3	M3 entegrasyon test desteği	Adım 3.9-3.12
BATUHAN	6	WAYPOINTS güncelleme	Faz 5
BATUHAN	6	MQ-2 eşik ince ayarı	Faz 5
EMRE	1	M4 geliştirme ortamı, model indirme	Bağımsız
EMRE	1	M4 YOLOv8n çıkarım boru hattı (mock)	Adım 2.6
EMRE	1	M6 FSM durum makinesi iskeleti	Bağımsız
EMRE	1	M6 füzyon skoru hesaplama	Adım 2.8
EMRE	2	M4 Pi performans testi	Adım 2.4

9. Toplantı Takvimi Özeti

Toplantı Gündem	Tür	Katılımcılar	Ne zaman
Faz 1 Kapanış Her kişi mock demo'sunu ekran paylaşımıyla gösterir. Tüm testler yeşil.	**Çevrimiçi**	**BATUHAN** **YUSUF** **EMRE** **SEMİH** **FURKAN** **BEKİR**	Faz 1 adımları tamamlandıktan sonra
Faz 2 Kapanış Pi SSH erişimi, 5 V doğrulaması, M4 FPS benchmark.	**Yüz yüze**	**ALPEREN** **HALİL** **EMRE**	Şasi hazır ve Pi açıkken
M5 API Gözden Geçirme M2/M3 API'leri anlatılır, kopyalama riski sıfırlanır.	**Çevrimiçi**	**FURKAN** **YUSUF** **BEKİR** **ALPEREN**	Faz 3 bitmeden hemen önce
Faz 3 Kapanış	**Yüz yüze**	**BEKİR** **ALPEREN** **BATUHAN**	Donanım testi tamamlandıktan sonra

Robot 100 cm gidiyor, 3 sensör çalışıyor, batarya okunuyor.			
Faz 4 Kapanış Engelli 3 sektör rotası, 6 snapshot.	**Yüz yüze**	**FURKAN** **YUSUF** **BEKİR**	Faz 4 tamamlandıktan sonra
M6 API Gözden Geçirme snapshot_callback arayüzü, M6 threading modeli netleştirilir.	**Çevrimiçi**	**EMRE** **FURKAN** **HALİL**	Faz 4 bittikten hemen sonra
Faz 5 Kapanış Tam ALARM döngüsü, SQLite canlı sorgu, snapshot dizini gösterimi.	**Yüz yüze**	**EMRE** **SEMİH** **FURKAN** **HALİL** **BEKİR** **BATUHAN**	Faz 5 tamamlandıktan sonra
Final Demo Provası 2 tam demo koşusu, zaman ölçümü, rol paylaşımı.	**Yüz yüze**	Tüm 8 kişi	Tüm fazlar bittikten sonra

10. Bağımlılık Özet Tablosu

Adım Neden	Kişi	Bloklayan
M4 Pi testi (2.9) Pi fiziksel olarak hazır olmalı	**EMRE**	Faz 2 Adım 2.4
M2 init_hardware (3.1) GPIO için Pi gerekli	**BEKİR**	Faz 2 kapanışı
M2 batarya (3.7) ADC okuma M3 üzerinden	**BEKİR**	M3 SPI adım 3.10
M3 HC-SR04 (3.9) GPIO + voltage divider hazır	**ALPEREN**	Faz 2 kapanışı
M3 MCP3208 (3.10) SPI wiring hazır	**BATUHAN**	Faz 2 kapanışı
M3 MLX90614 (3.13) I2C wiring hazır	**ALPEREN**	Faz 2 kapanışı
M5 fiziksel testler (4.x) M2+M3 çalışıyor olmalı	**FURKAN** **YUSUF**	Faz 3 kapanışı
M6 FSM + skor (Emre adımları) Laptop'ta bağımsız	**EMRE** **SEMİH**	Yok (mock yazılabilir)
M6 ALARM (5.1) JPEG + log zinciri	**HALİL**	M7 + M4 hazır
M6 M5 entegrasyon (5.2) FSM + M5 ikisi de hazır	**FURKAN**	FSM iskeleti + Faz 4

M6 mock entegrasyon (5.3) Geçiş birim testleri önce	**EMRE** **SEMİH**	FSM geçiş mantığı
M6 fiziksel test (5.5) Her şey entegre olmalı	Tüm M6 ekibi	5.1 + 5.2 + 5.3 + 5.4
Faz 6 (tüm) Demo hazırlığı	Çeşitli	Faz 5 kapanışı

11. Teknik Hatırlatmalar

Her modül sahibinin bilmesi gereken kritik noktalar:

1. **Arduino yok.** Motor PWM, encoder sayımı, HC-SR04 tetikleme, SPI ve I2C — bunların tamamı Raspberry Pi GPIO üzerinde çalışır. Arduino'ya gerek hissedilirse cevap hayırdır.
2. **config.py tek kaynak.** Herhangi bir sabit (pin numarası, eşik, dosya yolu, hız, zaman aşımı) doğrudan modül içine yazılmaz; config.py'e eklenir ve oradan import edilir.
3. **Mock Mode çalışmalı.** RPi.GPIO, spidev ve smbus2 yüklü olmayan bir bilgisayarda tüm modüller import ve çalışma hatası vermemeli. Donanım yokken simulation değerleri üretmeli.
4. **MCP3208 12-bit (0–4095).** Eski raporda 1023 değeri hatalıydı. Füzyon skorundaki smoke paydasında ve tüm eşik karşılaştırmalarında 4095 kullanılır.
5. **Batarya eşikleri 2S 18650 içindir.** Uyarı: 6.8 V, kritik: 6.4 V. Eski 3S değerleri (9.6 V, 8.4 V) geçersizdir.
6. **HC-SR04 gerilim bölücüler zorunludur.** 3 sensör, 3 bölücü (her biri 1kΩ + 2kΩ). Echo pinini direkt Pi GPIO'ya bağlamak pini kalıcı olarak bozar.
7. **Encoder ile geri dönüş sırasında sensör kullanılmaz.** Yan geçiş sonrası batı yönüne dönerken sol sensör güneye bakar; oradaki herhangi bir engel yanlış erken durdurma üretir. Encoder ile toplam yan mesafe geri alınır, sensör kullanılmaz.
8. **Sektör orta nokta snapshot'ı hiçbir koşulda atlanamaz.** `midpoint_callback` hem ana navigasyon döngüsünden hem de engel kaçınma yan geçiş döngüsünden çağrılır. Detour rotasında bile orta nokta geçildiğinde snapshot tetiklenir.

DOKÜMAN SONU

SeeFire — Grup 4 — CSE 396 — 2025–2026 Bahar

Paralel Çalışma Planı — Rapor v2.0 ile uyumlu